COMPUTER NETWORKS PROJECT

SPIKE

Wireless Communication System

Submitted By

Hanzla Bin Imran — 2025-CS-748

Farrukh Waseem Khan — 2025-CS-745

Ali Raza — 2025-CS-738

Alauddin — 2025-CS-760

Submitted To

Ma'am Ghazala

Table of Contents

Table of Contents 2
Abstract 4
1. Introduction 5
2. Problem Statement 5
3. Project Objectives 5
4. Scope of the Project 6
5. Technologies and Tools Used 7
6. System Architecture 7
7. Server-Side Implementation 10
7.1 Connection Handling and Multithreading 10
7.2 Message Routing 10
7.3 Media Relay (Port 5051) 10
7.4 Error Handling (Server) 11
8. Client-Side Implementation 11
8.1 Login 11
8.2 Main Interface 11
8.3 File Sharing 11
8.4 Voice and Video Calls 11
8.5 Error Handling (Client) 12
9. Communication Protocols and Data Flow 12
9.1 Control Protocol (JSON over TCP, Port 5050) 12
9.2 Media Protocol (Struct-Framed Packets, Port 5051) 12
9.3 End-to-End Flow of a Private Message 12
10. Installation and Configuration 13
10.1 Prerequisites 13
10.2 Project Files 13
10.3 Installing Python Packages 13
10.4 Firewall Configuration 13
11. Execution and Wireless Network Setup 14
11.1 Finding the Server's Wireless IPv4 Address 14
11.2 Starting the Server 14
11.3 Starting a Client and Connecting Another Computer 14
11.4 Network Setup Checklist 15

12. Testing and Results 16
13. Security and Privacy Considerations 17
14. Limitations 18
15. Future Enhancements 18
16. Conclusion 18
17. References 19

Abstract

Spike is a Python-based wireless communication system that allows multiple computers connected through the same reachable Wi-Fi network (or any reachable local IP network) to exchange messages, share files, and hold voice and video calls in real time. The application follows a client–server architecture: a single server program (server.py) coordinates all connected clients, routes chat traffic, manages group chat rooms, relays media during calls, and tracks which users are online.

Client applications (client.py) provide a graphical user interface built with Tkinter, through which users log in with a unique display name, send private messages, create and join chat rooms, transfer files to other users or room members, and initiate voice or video calls. Communication is built on Python sockets over TCP: a chat/control channel on port 5050 carries structured JSON messages, while a separate media relay on port 5051 carries struct-framed audio and video packets between call participants.

This document presents the complete technical documentation of Spike, including its architecture, server-side and client-side implementation, communication protocols, installation and execution procedures, wireless network setup, testing plan, security considerations, limitations, and proposed future enhancements.

1. Introduction

Modern computer networks make it possible for devices to communicate almost instantly over shared infrastructure. A wireless local area network (WLAN) is one of the most common such infrastructures: every laptop and smartphone within range of the same Wi-Fi router shares a common IP network and can, in principle, talk directly to every other device on that network.

Spike turns this capability into a practical, user-friendly application. It is a Python-based wireless communication system that enables several computers on the same reachable Wi-Fi network (or another reachable local IP network) to communicate with one another using a client–server model. Each participant runs the Spike client, logs in with a unique display name, and can immediately begin private or group conversations, share files, and make voice or video calls — all without requiring Internet access or any external service.

The project demonstrates core computer networking concepts in a working system: socket programming, TCP reliable transport, multithreaded server design, application-layer message protocols, message routing, and the separation of control and media traffic onto distinct channels.

2. Problem Statement

Small teams, classrooms, labs, and households frequently need quick communication between computers that are already connected to the same Wi-Fi router. Existing solutions either depend on Internet connectivity (messaging platforms, VoIP services), require complex installation (self-hosted chat servers with databases and web front ends), or offer only a subset of the required features.

There is a need for a lightweight, self-contained communication system that:

- works entirely within a local Wi-Fi network, with no Internet dependency;
- supports real-time private messaging and group chat rooms;
- allows files to be transferred between users;
- supports voice and video calls between connected computers; and
- provides a simple graphical interface so that non-technical users can operate it.

Spike addresses this need using only the Python standard library plus a small number of common third-party packages.

3. Project Objectives

1. Implement a multithreaded TCP server capable of handling many simultaneous clients.
2. Provide user login with unique display names and maintain a live online-user list.
3. Support real-time private (one-to-one) messaging.
4. Support group chat rooms, including room creation, joining, and leaving.
5. Enable file sharing between individual users and room members using Base64 encoding.
6. Implement voice calling and video calling with a dedicated media relay channel.
7. Deliver a graphical user interface built with Tkinter that exposes all features clearly.

8. Demonstrate correct separation of control traffic (JSON on port 5050) and media traffic (struct-framed packets on port 5051).
9. Document the system thoroughly for evaluation and future maintenance.

4. Scope of the Project

The scope of Spike is defined precisely by its current implementation, and this documentation does not claim functionality beyond it.

In scope (implemented):

- Multi-client chat over a local Wi-Fi / reachable IP network.
- Login with unique display names; duplicate names are not allowed.
- Private messaging between online users.
- Chat rooms: create, join, leave, and broadcast room messages.
- File sharing with users and room members via Base64-encoded transfer.
- Voice and video calls, with call signaling over the control channel and media relayed over a separate TCP port.
- Online user list and connection status displayed in the GUI.
- Tkinter-based graphical interface; OpenCV-based camera capture and video display; NumPy and SoundDevice for audio/video processing.

Out of scope (not implemented in the supplied code):

- Internet-based communication (the system works over Wi-Fi or another reachable local IP network only; it does not automatically work over the Internet).
- Encryption of messages or media streams.
- Database storage or persistent accounts (login state exists only for the duration of a session).
- Advanced authentication such as passwords or tokens.
- File transfer resume, offline message delivery, or message history persistence.

5. Technologies and Tools Used

Technology / Tool	Role in Spike
Python 3	Implementation language for both server.py and client.py.
Python sockets (socket module)	TCP communication between clients and the server on ports 5050 and 5051.
threading module	Multithreading: the server runs one handler thread per client; the client runs separate threads for receiving control messages and capturing/sending media.
JSON (json module)	Format of all chat/control messages exchanged on port 5050.
Base64 (base64 module)	Encoding of file contents for sharing over the control channel.
Tkinter	Graphical user interface of the client application.
OpenCV (cv2)	Camera capture and video frame display during video calls.
NumPy	Numerical handling of audio/video frame data.
SoundDevice	Audio capture and playback for voice/video calls.
struct module	Binary framing of media packets exchanged over the media relay (port 5051).

Table 1: Technologies and tools used in Spike.

Development can be carried out in any editor or IDE (e.g., VS Code, PyCharm); no specific tool is required by the code.

6. System Architecture

Spike uses a client–server architecture. One computer on the Wi-Fi network runs server.py, which acts as the central coordinator. Every other computer runs client.py and connects to the server over the local network.

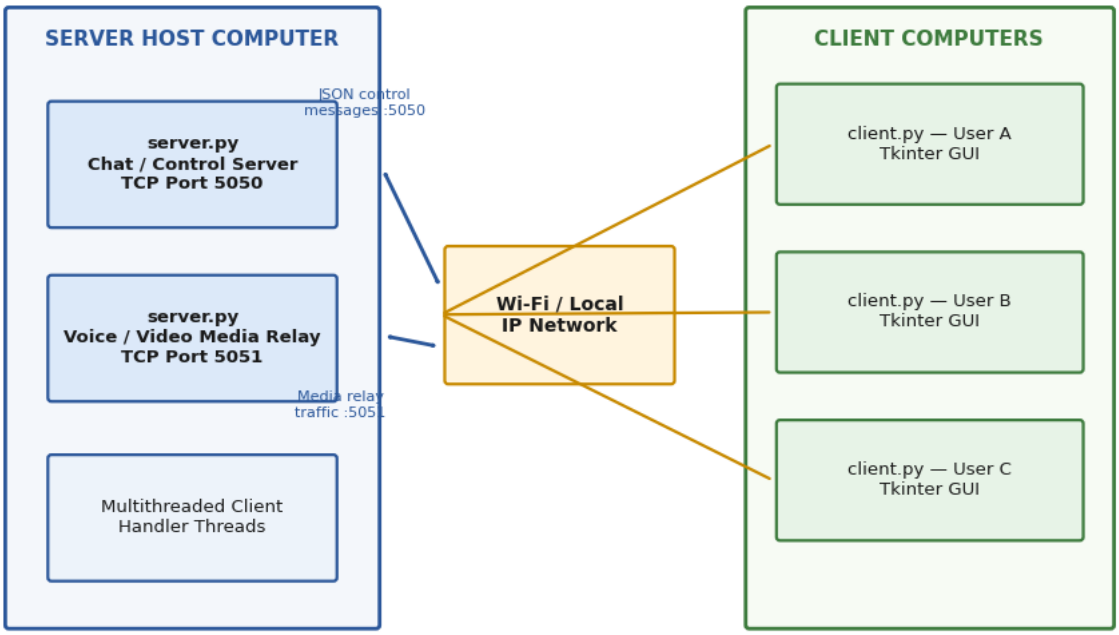


Figure 1: System architecture of Spike — clients connect over Wi-Fi to the control server (port 5050) and the media relay (port 5051).

The design separates traffic onto two TCP channels:

- **Control channel (port 5050):** login, private messages, room management, file-sharing requests and payloads, call signaling, and presence (online list). All control messages are JSON objects.
- **Media channel (port 5051):** raw audio and video frames relayed between call participants during voice/video calls. Frames are framed using Python's struct module for length-prefixed packet delivery.

This separation keeps text chat responsive even while media streams are active, and mirrors the control/media split used by protocols such as SIP/RTP in real-world VoIP systems.

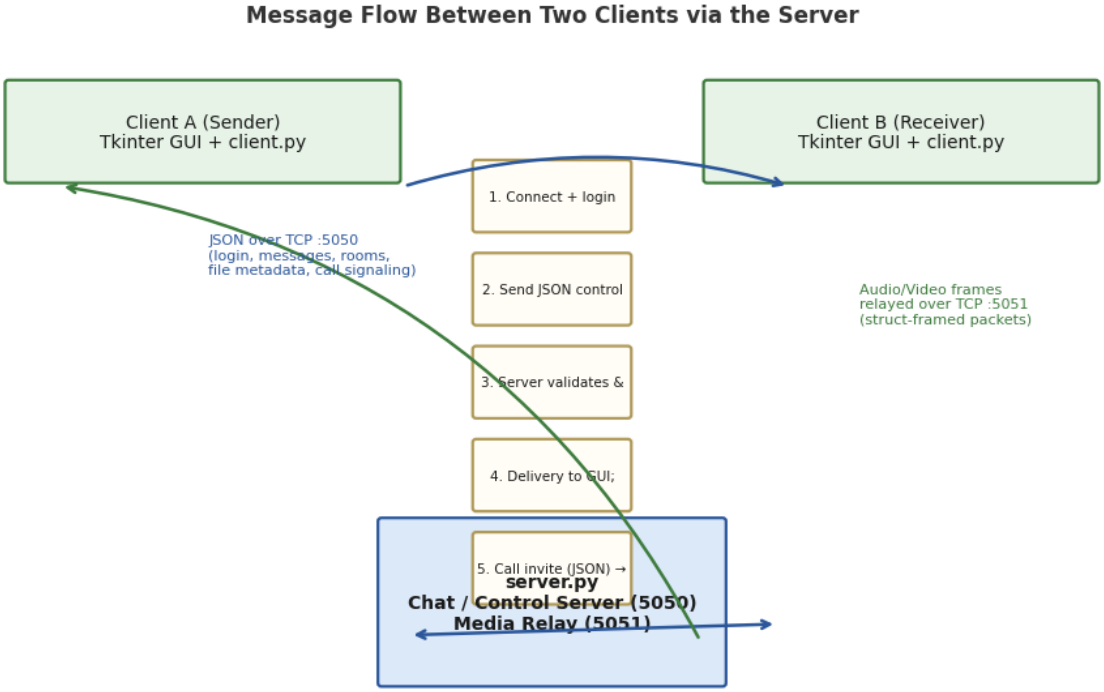


Figure 2: Simplified communication flow between two clients through the server.

7. Server-Side Implementation

The server (server.py) is the single point of coordination in Spike. Its main responsibilities are accepting client connections, maintaining session state, routing messages, managing rooms, and relaying call media.

7.1 Connection Handling and Multithreading

The server creates a TCP socket, binds it to port 5050 on the host machine, and listens for incoming client connections. For each client that connects, the server spawns a dedicated handler thread. This multithreaded design allows the server to serve many clients concurrently: one client's activity never blocks the others.

- Maintains a registry of connected clients and their unique display names.
- Rejects or prompts again when a login name is already in use.
- Detects disconnections (closed sockets / failed sends) and removes the client from the registry and any rooms.

7.2 Message Routing

Every message received on the control channel is parsed as JSON. Based on the message type field, the server routes it appropriately:

Message Type (conceptual)	Server Action
login	Validate the requested display name; register the client; broadcast the updated online list.
private message	Forward the message to the named recipient only.
create room	Create a new chat room and register its creator as a member.
join room / leave room	Update room membership; notify members as appropriate.
room message	Broadcast the message to all current members of the room.
file request / file data	Route file-sharing requests and Base64-encoded file content between users or to room members.
call invite / call accept / call end	Relay call signaling between caller and callee to establish, accept, or tear down voice/video calls.
disconnect	Remove the client, update rooms and the online list, and notify other clients.

Table 2: Control messages handled by the server and their routing behavior.

7.3 Media Relay (Port 5051)

For voice and video calls, the server runs a separate TCP listener on port 5051. After call signaling on the control channel establishes a call between two clients, audio and video frames are forwarded through this media relay so that the call participants can exchange media streams

without interfering with chat traffic on port 5050. Media packets are length-prefixed using the struct module so the receiver can reconstruct complete frames from the TCP byte stream.

7.4 Error Handling (Server)

- Socket and JSON parsing failures are caught so a single malformed message cannot crash the server.
- Broken client connections are detected and cleaned up, including room membership and the online list.
- Failed sends (e.g., to a client that just disconnected) result in graceful removal rather than a server crash.

8. Client-Side Implementation

The client (client.py) provides the user-facing application. It connects to the server's control port, presents a Tkinter interface, and manages all user interactions.

8.1 Login

On startup, the client shows a login screen asking for a display name. The name is sent to the server; if it is already taken, the user is asked to choose another. Once accepted, the main interface opens and the user's name appears in the online list.

8.2 Main Interface

- Online user list with live connection status.
- Conversation area for private messages with a selected user.
- Room list with options to create, join, leave, and chat in rooms.
- Controls for sending files and initiating voice/video calls.

8.3 File Sharing

When a user sends a file, the client reads the file's contents, encodes them with Base64, and transmits the encoded data as JSON control messages. The recipient decodes the Base64 data back into the original file bytes. The same mechanism works for direct user-to-user transfers and for sharing with all members of a room.

8.4 Voice and Video Calls

Voice and video calls proceed in two phases:

10. **Signaling (port 5050):** the caller sends a call invitation as a JSON control message; the callee accepts (or the call is ended). The server relays these messages between the two parties.
11. **Media (port 5051):** after acceptance, the clients capture audio using SoundDevice (and video frames using OpenCV for video calls), and the frames are relayed through the server's media channel. Incoming frames are decoded and played back / displayed in the GUI. NumPy is used to process the raw frame data.

8.5 Error Handling (Client)

- Connection loss to the server is reported in the interface instead of failing silently.
- Malformed incoming messages are ignored safely.
- Call and camera/audio device errors are surfaced to the user where the code handles them.

9. Communication Protocols and Data Flow

9.1 Control Protocol (JSON over TCP, Port 5050)

All chat/control traffic consists of JSON objects sent over a TCP connection. TCP provides reliable, ordered delivery, which suits control messages. The exact field set used by the implementation is defined in the source code; conceptually, each message contains a type field identifying the operation and payload fields carrying the data.

Illustrative example (based on the message types handled by the implementation):

```
{
  "type": "private_message",
  "from": "Hanzla",
  "to": "Ali Raza",
  "message": "Hello, are you online?"
}
```

Example of a JSON control message exchanged on port 5050 (structure shown for documentation; field names follow the implementation).

9.2 Media Protocol (Struct-Framed Packets, Port 5051)

Raw audio and video frames are sent over the second TCP connection using length-prefixed framing built with the struct module: a fixed-size header carries the frame length, followed by the frame payload. The receiver reads the header, then reads exactly that many bytes to obtain a complete frame, and passes it to the audio (SoundDevice) or video (OpenCV) playback path.

9.3 End-to-End Flow of a Private Message

12. Client A types a message and selects Client B in the online list.
13. Client A builds a JSON control message and sends it to the server on port 5050.
14. The server looks up Client B's socket and forwards the message.
15. Client B's receiver thread parses the JSON and displays the message in the GUI.
16. The server (and clients) keep the online list and room memberships up to date on connect/disconnect.

10. Installation and Configuration

10.1 Prerequisites

- Python 3.x installed on the server computer and on every client computer.
- All computers connected to the same Wi-Fi network (or otherwise mutually reachable IP network).

10.2 Project Files

The project folder contains at minimum the two runnable programs:

File	Purpose
server.py	Chat/control server (port 5050) and voice/video media relay (port 5051).
client.py	Tkinter client application with messaging, rooms, file sharing, and voice/video calls.

Table 3: Spike project files.

10.3 Installing Python Packages

Install the required third-party packages on each client computer (and the server, if run on a separate machine) using pip:

```
pip install opencv-python numpy sounddevice
```

Terminal command to install Spike's third-party dependencies.

Tkinter normally ships with Python; if it is missing, install it via your operating system's package manager (for example, sudo apt install python3-tk on Ubuntu/Debian).

10.4 Firewall Configuration

The operating system's firewall on the server computer must allow inbound TCP connections on the two ports used by Spike. Examples:

```
# Windows (run in an Administrator command prompt or PowerShell):
netsh advfirewall firewall add rule name="Spike Control" dir=in action=allow
protocol=TCP localport=5050
netsh advfirewall firewall add rule name="Spike Media" dir=in action=allow
protocol=TCP localport=5051
```

Windows firewall rules for Spike.

```
# Linux (ufw example):
sudo ufw allow 5050/tcp
sudo ufw allow 5051/tcp
```

Linux firewall rules for Spike.

If either rule is missing, clients will be unable to connect or media streams will fail, so this step is essential before testing on multiple computers.

11. Execution and Wireless Network Setup

11.1 Finding the Server's Wireless IPv4 Address

Each client must connect to the server computer's IPv4 address on the Wi-Fi network. On the server machine, find the wireless adapter's address:

```
# Windows:
ipconfig
# Look for the "Wireless LAN adapter Wi-Fi" section and its IPv4 Address

# Linux / macOS:
ip addr          # Linux
ifconfig         # macOS
```

Commands to find the server's wireless IPv4 address.

11.2 Starting the Server

On the server computer, open a terminal in the project folder and run:

```
python server.py
```

Starting the Spike server.

The server starts listening for control connections on port 5050 and media relay connections on port 5051. Leave this terminal open for as long as the system is in use.

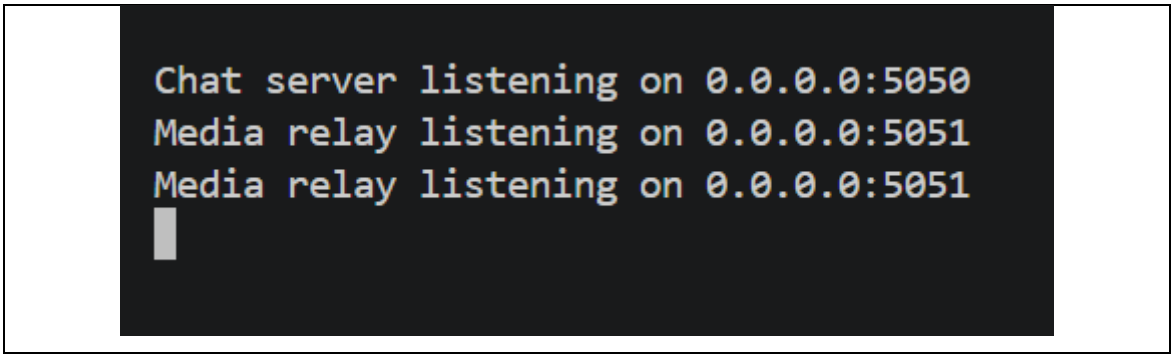


Figure 3: Genuine screenshot placeholder — server terminal showing server.py running and listening on ports 5050/5051.

11.3 Starting a Client and Connecting Another Computer

On each client computer, open a terminal in the project folder and run:

```
python client.py
```

Starting the Spike client.

When the client starts, enter the server's wireless IPv4 address (from Section 11.1) and a unique display name. Repeat on every additional computer, giving each user a different display name. All connected users then appear in the online list and can message, share files, and call one another.

11.4 Network Setup Checklist

- Server and all clients are connected to the same Wi-Fi network (or mutually reachable network).
- server.py is running and its terminal shows successful startup.
- Firewall on the server allows TCP 5050 and 5051 inbound.
- Each client enters the server's correct wireless IPv4 address.
- Each user logs in with a unique display name.

12. Testing and Results

The following test plan covers the implemented functionality of Spike. Actual results are intentionally left unconfirmed (marked Pending) until the tests are executed on real machines over Wi-Fi; after testing, record the observed behavior and set the status to Pass or Fail.

#	Test Case	Expected Result	Actual Result	Status
T01	Start server.py on the server computer	Server starts and listens on ports 5050 and 5051 without errors.	Pending	Pending
T02	Start client.py and log in with a unique display name	Login accepted; main interface opens; user appears in the online list.	Pending	Pending
T03	Attempt login with a name already in use	Server rejects the duplicate; client asks for a different name.	Pending	Pending
T04	Connect a second client and verify the online list	Both users appear in each other's online list with correct status.	Pending	Pending
T05	Send a private message between two clients	Message is delivered in real time to the correct recipient only.	Pending	Pending
T06	Create a chat room	Room is created; creator becomes a member; room appears in the room list.	Pending	Pending
T07	Second client joins the room and sends a room message	Message is broadcast to all room members; non-members do not receive it.	Pending	Pending
T08	Leave the room	Client is removed from membership; room members are updated.	Pending	Pending
T09	Send a file to another user	File is Base64-encoded, transferred, and reconstructed correctly at the recipient.	Pending	Pending
T10	Share a file with	All room members	Pending	Pending

	room members	receive the file data.		
T11	Initiate a voice call	Call invitation is signaled; on acceptance, audio is captured and relayed (port 5051) and heard at both ends.	Pending	Pending
T12	Initiate a video call	Camera frames are captured (OpenCV) and displayed at both ends; audio works as in T11.	Pending	Pending
T13	End a call	Media streams stop; both clients return to normal chat state.	Pending	Pending
T14	Disconnect a client abruptly (close program)	Server detects the drop and updates the online list and room memberships.	Pending	Pending
T15	Block port 5051 on the server firewall and attempt a call	Call signaling still works but media relay fails, confirming the two-channel design.	Pending	Pending

Table 4: Spike test plan. Actual results and statuses remain pending until testing is performed.

13. Security and Privacy Considerations

This section describes the security posture of Spike exactly as implemented, without overstating it.

- **No encryption:** messages, file contents, and media streams travel over the network in plain form (JSON/Base64 text and raw framed media). Base64 is an encoding, not encryption, and provides no confidentiality.
- **Local network boundary:** Spike is designed for a trusted Wi-Fi / local IP network. Since traffic is unencrypted, the network itself should be trusted (e.g., a private router), because anyone able to capture local traffic could read it.
- **Weak authentication:** login uses only a unique display name. There are no passwords or tokens, so a user could claim an existing name if the timing allowed it. This is acceptable for a lab/demo setting but not for production use.
- **No persistence:** no database or accounts exist; all state lives in memory on the server for the duration of the session. This limits exposure of stored data but means there is no audit trail.

- **Input robustness:** the server catches malformed JSON and broken sockets so malformed input cannot trivially crash the service; however, no authentication or authorization beyond name uniqueness is enforced on control messages.

14. Limitations

- Operates over Wi-Fi or another reachable local IP network only; it does not work over the Internet automatically (would require port forwarding or a public address — see Future Enhancements).
- No encryption or integrity protection for any traffic.
- No persistent accounts, message history, or offline message delivery; everything is session-based.
- File sharing uses Base64 within JSON control messages, which adds size overhead (about 33 percent) and can be slow for large files.
- Performance depends on the Wi-Fi network quality; video calling in particular requires adequate bandwidth and low contention.
- The media relay routes through the server, adding latency and load on the server machine during calls.
- Error handling covers connection and parsing failures, but there is no automatic reconnection or call-quality adaptation.

15. Future Enhancements

The following are proposed improvements — none of them is currently implemented, and they are listed separately from the implemented functionality.

17. **Encryption:** end-to-end encryption of control messages and media streams (e.g., TLS on both ports, or a key exchange for media).
18. **Persistence:** a lightweight database (e.g., SQLite) for users, credentials, and chat history.
19. **Internet operation:** Internet reachability via secure tunneling or a hosted relay, instead of manual port forwarding.
20. **File transfer:** efficient binary file transfer with chunking, progress display, and resume support, replacing Base64-in-JSON.
21. **P2P media:** direct peer-to-peer media paths (NAT traversal / local P2P) to reduce server load and latency.
22. **Group calls:** group voice/video calls with multiple participants.
23. **Reliability:** automatic reconnection, offline message queueing, and unread-message indicators.
24. **Packaging:** cross-platform packaging (e.g., PyInstaller executables) so end users need not install Python.

16. Conclusion

Spike demonstrates how a small set of Python building blocks — sockets, threads, JSON, Tkinter, and a handful of media libraries — can be assembled into a complete wireless communication

system. The two-channel design (JSON control on TCP 5050, struct-framed media on TCP 5051) mirrors production VoIP practice and keeps the system organized and extensible.

Within its intended environment — computers sharing a reachable Wi-Fi network — Spike delivers user login with unique display names, real-time private and room messaging, file sharing, voice and video calls, and live presence information through a friendly graphical interface. Its known limitations (no encryption, no persistence, local-network scope) are documented honestly and mapped to concrete future enhancements. The project successfully meets its objectives as a university computer networks project and provides a solid foundation for further development.

17. References

Python Software Foundation. socket — Low-level networking interface. Python documentation.

Python Software Foundation. json — JSON encoder and decoder. Python documentation.

Python Software Foundation. base64 — Base16, Base32, Base64, Base85 data encodings. Python documentation.

Python Software Foundation. struct — Interpret bytes as packed binary data. Python documentation.

Python Software Foundation. threading — Thread-based parallelism. Python documentation.

Python Software Foundation. tkinter — Python interface to Tcl/Tk. Python documentation.

OpenCV team. OpenCV-Python Tutorials (camera capture and image display).

NumPy developers. NumPy documentation (array handling for media data).

SoundDevice project documentation (audio capture and playback).

Tanenbaum, A. S., and Wetherall, D. Computer Networks (5th ed.) — client–server model, TCP, and application-layer protocols.

Course lecture material on socket programming and transport-layer protocols.